

Multi-Pipeline ETL and Reporting Framework

Code Review and Project Report

Group 4

Trupti Khodwe(IMT2022007)

Rishabh Dixit(IMT2022051)

Priyanshu Tiwari(IMT2023010)

Siddharth Kini(IMT2023026)

Github Link:

<https://github.com/truptikhodwe/Multi-Pipeline-ETL-and-Reporting-Framework-for-Web-Server-Log-Analytics>

Demo video link: [Demo Video Link](#)

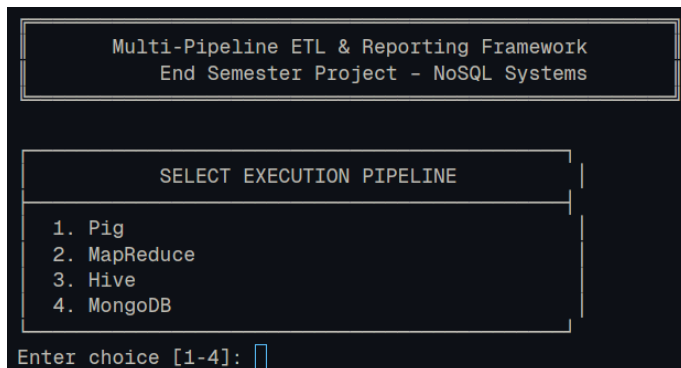
1. Overview

This project is a command-line ETL prototype for NASA web server logs. The same analytics workload can be run through four execution backends: Pig, MapReduce, Hive, and MongoDB. The results are loaded into PostgreSQL and displayed via a single common report generator.

The code uses a single main entry point, four pipeline controllers, and a reporting layer. This keeps the whole system consistent even though the backend changes.

2. Architecture

The system starts with the NASA HTTP access logs from July 1995 and August 1995. The script treats these as two batches. The user then selects a pipeline, a query, and a batch from the CLI.



- Main.java serves as the single entry point and delegates work to PigPipeline, MapReducePipeline, HivePipeline, or MongoDBPipeline.
- Each backend performs the ETL work and writes the final results to PostgreSQL.
- The ReportGenerator reads the saved results and prints a final report.
- The design maintains the same query logic and reporting format across all pipelines.

3. Parsing and normalization

All pipelines use the same basic log-parsing approach. A regex is used to pull out the host, timestamp, request, status code, and byte count from each log line. The timestamp is then split into date and hour, and the request string is split into method, resource path, and protocol version.

If a line does not match the expected format, it is counted as malformed rather than ignored. If the byte field is missing or shown as '-', it is treated as 0.

- Pig uses REGEX_EXTRACT and filtering to keep only valid rows.
- Hive uses RegexSerDe for the raw table and then creates a cleaned table.
- MapReduce uses a cleaner job before the query jobs.
- MongoDB uses Java parsing before inserting cleaned documents.

4. ETL workflow by pipeline

4.1 Pig

The Pig pipeline uses a single Pig Latin script for cleaning and analytics. The raw logs are loaded, parsed, cleaned, and then written for the three queries.

- Query 1 groups by date and status code and calculates request count and total bytes.
- Query 2 groups by resource path, counts requests, sums bytes, counts distinct hosts, sorts by request count, and keeps the top 20.
- Query 3 separates total requests and error requests, joins the results, and calculates the error rate.

4.2 MapReduce

The MapReduce pipeline first cleans the raw logs and then runs the selected query job on the cleaned output. The cleaner job handles parsing and tracking malformed records.

- Query 1 emits date and status-code pairs, then sums the request count and bytes.
- Query 2 counts requests, total bytes, and distinct hosts for each resource path.
- Query 3 filters status codes in the 400-599 range and calculates error statistics.

4.3 Hive

The Hive pipeline uses SQL-style queries and RegexSerDe to parse the raw log files. It creates a raw table and a cleaned table, then runs the three queries from HQL scripts.

- Query 1 groups by date and status code.
- Query 2 uses distinct host counting and limits the result to the top 20 resources.
- Query 3 computes total requests, error requests, error rate, and distinct error hosts.

4.4 MongoDB

The MongoDB pipeline uses Java to read and clean the logs, then uses MongoDB aggregation pipelines for the queries. The cleaned records are stored as documents and then processed with aggregation stages.

- Query 1 groups by date and status code.
- Query 2 uses grouping, unique host collection, and sorting by request count.
- Query 3 computes error counts and error rates using aggregation logic.

5. Query implementation and output semantics

The three mandatory queries are the same across all pipelines. The output columns remain the same regardless of which backend is used.

```
SELECT QUERY

1. Query 1 - Daily Traffic Summary
2. Query 2 - Top 20 Requested Resources
3. Query 3 - Hourly Error Analysis
4. All Queries

Enter choice [1-4]: 
```

- Query 1: log date, status code, request count, total bytes.
- Query 2: resource path, request count, total bytes, and distinct host count.
- Query 3: log date, log hour, error request count, total request count, error rate, and distinct error hosts.

The script also shows that batch information is carried through the workflow so the reporting layer can separate results from Batch 1 and Batch 2.

6. Batching and runtime reporting

```
SELECT BATCH / DATASET

1. Batch 1 - July 1995 logs
2. Batch 2 - August 1995 logs
3. All batches (combined)

Enter choice [1-3]: 
```

The script treats the July 1995 and August 1995 log files as two batches. Batch 1 is the July file, and Batch 2 is the August file.

Batch IDs are shown in the CLI flow and in the stored results. The report also shows runtime, batch size, average batch size, records processed, and the number of malformed records.

The average batch size is described as total records processed divided by the number of batches processed.

7. PostgreSQL result schema

The PostgreSQL database named `etl_project` stores the results of every run. The schema is simple and easy to report from.

```

postgres=# \c etl_project
You are now connected to database "etl_project" as user "postgres".
etl_project=# \d

```

List of relations			
Schema	Name	Type	Owner
public	batch_metadata	table	postgres
public	batch_metadata_id_seq	sequence	postgres
public	malformed_records_summary	table	postgres
public	malformed_records_summary_id_seq	sequence	postgres
public	query1_results	table	postgres
public	query1_results_id_seq	sequence	postgres
public	query2_results	table	postgres
public	query2_results_id_seq	sequence	postgres
public	query3_results	table	postgres
public	query3_results_id_seq	sequence	postgres
public	runs	table	postgres
public	runs_run_id_seq	sequence	postgres

```

(12 rows)

```

- runs stores the main run metadata, such as pipeline name, query name, batch ID, batch size, average batch size, records processed, malformed record count, runtime, and execution time.
- batch_metadata stores information about the batch source and record count.
- malformed_records_summary stores malformed record totals and related statistics.
- query1_results, query2_results, and query3_results store the final outputs for the three queries.

8. Major design decisions

- Use one CLI so the user can switch between pipelines without changing the workflow.
- Keep the parsing and query logic aligned across all four backends.
- Load the final results into PostgreSQL to maintain consistent reporting across pipelines.
- Use simple file- and batch-based handling so the demo is easy to show and explain.

9. Comparison of the four pipelines

All four pipelines aim to answer the same questions, but they differ in style.

- Pig is simple to write for data flow tasks, especially when the logic fits a script.
- MapReduce is the most verbose, but it clearly shows the ETL steps.
- Hive is convenient for SQL-like analytics and is easy to read.
- MongoDB is flexible and well-suited to document-based processing and aggregation.

In terms of correctness, all four should produce the same logical results because they follow the same parsing rules and query definitions. In terms of runtime and ease of implementation, the pipelines differ, but the shared reporting layer makes comparison straightforward.

Pipeline	Correctness	Runtime	Difficulty of Implementation	Suitability for Semi-Structured Log Analytics

MongoDB	High: results match other pipelines with negligible difference in malformed records	~61.7 s (fastest) MongoDB offers schema flexibility and a faster aggregation framework.	Moderate: requires schema design, indexing, aggregation pipelines, and ETL scripting	Very High: flexible document model is naturally suited for semi-structured logs
Pig	High: outputs consistent with Hive and MapReduce	~288.7 s Pig Latin scripts are converted into MR jobs. Additional layers for data-flow management and ETL abstractions.	Low to Moderate: high-level scripting simplifies ETL and aggregation tasks	High: Pig Latin handles semi-structured data conveniently with less boilerplate
MapReduce	High: produces identical analytical results	~112.2 s Provide low-level control over execution to enable manual optimization in Java.	High: requires verbose Java code and manual implementation of parsing and aggregation logic	Moderate: powerful but cumbersome for evolving semi-structured data
Hive	High: same analytical output as other systems	~270.0 s HiveQL queries are translated internally into MapReduce jobs, introducing additional overheads.	Low: SQL-like interface makes querying straightforward	Very High: ideal for large-scale log analytics with structured querying and partitioning support

Below are the ETL run reports for each of the four pipelines:

ETL RUN REPORT

Run ID : 4

Pipeline : MapReduce

Query : All

Batch ID : 1

Avg Batch Size : 1730806.00

Records Processed : 3461612

Malformed Records : 8313

Runtime : 112230 ms

Executed At : 2026-05-17 11:37:02.243702

-- Malformed Records Summary --

Malformed : 8313 / 3461612 (0.24%)

-- Query 1: Daily Traffic Summary (first 10 rows) --

Date	Status	Requests	Total Bytes
01/Aug/1995	200	30657	523879212
01/Aug/1995	302	399	29856
01/Aug/1995	304	2608	0
01/Aug/1995	403	1	0
01/Aug/1995	404	242	0
01/Jul/1995	200	57846	1605426462
01/Jul/1995	302	2566	218397
01/Jul/1995	304	3797	0
01/Jul/1995	404	314	0
02/Jul/1995	200	54339	1521111456

-- Query 2: Top Requested Resources (first 10) ---

Resource	Requests	Bytes	Hosts
/images/NASA-logosmall.gif	111086	70379226	49583
/images/NASA-logosmall.gif	97267	61055694	41018
/images/KSC-logosmall.gif	89529	92472016	49049
/images/KSC-logosmall.gif	75278	76688780	44912
/images/MOSAIC-logosmall.gif	67349	21158907	30981
/images/USA-logosmall.gif	66968	13561002	30766
/images/WORLD-logosmall.gif	66344	38492922	30488
/images/ksclogo-medium.gif	62663	319890578	26968
/images/MOSAIC-logosmall.gif	60299	19338462	29729
/images/USA-logosmall.gif	59844	12369240	29490

-- Query 3: Hourly Error Analysis (first 10 rows) --

Date	Hour	Errors	Total	Error%	Hosts
01/Aug/1995	0	4	1635	0.24%	4
01/Aug/1995	1	18	1365	1.32%	7
01/Aug/1995	2	5	986	0.51%	2
01/Aug/1995	3	85	1217	6.98%	16
01/Aug/1995	4	1	985	0.10%	1
01/Aug/1995	5	5	1145	0.44%	4
01/Aug/1995	6	1	1020	0.10%	1
01/Aug/1995	7	9	1778	0.51%	6
01/Aug/1995	8	30	2857	1.05%	11
01/Aug/1995	9	11	3199	0.34%	7

Hive

ETL RUN REPORT

Run ID : 5

Pipeline : Hive

Query : All

Batch ID : 1

Avg Batch Size : 1730806.00

Records Processed : 3461612

Malformed Records : 8313

Runtime : 269516 ms

Executed At : 2026-05-17 11:43:51.840636

-- Malformed Records Summary --

Malformed : 8313 / 3461612 (0.24%)

-- Query 1: Daily Traffic Summary (first 10 rows) --

Date	Status	Requests	Total Bytes
01/Aug/1995	200	30657	523879212
01/Aug/1995	302	399	29856
01/Aug/1995	304	2608	0
01/Aug/1995	403	1	0
01/Aug/1995	404	242	0
01/Jul/1995	200	57846	1605426462
01/Jul/1995	302	2566	218397
01/Jul/1995	304	3797	0
01/Jul/1995	404	314	0
02/Jul/1995	200	54339	1521111456

-- Query 2: Top Requested Resources (first 10) ---

Resource	Requests	Bytes	Hosts
/images/NASA-logosmall.gif	111086	70379226	49583
/images/NASA-logosmall.gif	97267	61055694	41018
/images/KSC-logosmall.gif	89529	92472016	49049
/images/KSC-logosmall.gif	75278	76688780	44912
/images/MOSAIC-logosmall.gif	67349	21158907	30981
/images/USA-logosmall.gif	66968	13561002	30766
/images/WORLD-logosmall.gif	66344	38492922	30488
/images/ksclogo-medium.gif	62663	319890578	26968
/images/MOSAIC-logosmall.gif	60299	19338462	29729
/images/USA-logosmall.gif	59844	12369240	29490

-- Query 3: Hourly Error Analysis (first 10 rows) --

Date	Hour	Errors	Total	Error%	Hosts
01/Aug/1995	0	4	1635	0.24%	4
01/Aug/1995	1	18	1365	1.32%	7
01/Aug/1995	2	5	986	0.51%	2
01/Aug/1995	3	85	1217	6.98%	16
01/Aug/1995	4	1	985	0.10%	1
01/Aug/1995	5	5	1145	0.44%	4
01/Aug/1995	6	1	1020	0.10%	1
01/Aug/1995	7	9	1778	0.51%	6
01/Aug/1995	8	30	2857	1.05%	11
01/Aug/1995	9	11	3199	0.34%	7

MapReduce

ETL RUN REPORT

Run ID : 1

Pipeline : MongoDB

Query : All

Batch ID : 1

Avg Batch Size : 1730806.50

Records Processed : 3461613

Malformed Records : 8314

Runtime : 61724 ms

Executed At : 2026-05-17 11:24:46.584582

Malformed Records Summary

Malformed : 8314 / 3461613 (0.24%)

Query 1: Daily Traffic Summary (first 10 rows)

Date	Status	Requests	Total Bytes
01/Aug/1995	200	30657	523879212
01/Aug/1995	302	399	29856
01/Aug/1995	304	2608	0
01/Aug/1995	403	1	0
01/Aug/1995	404	242	0
01/Jul/1995	200	57846	1605426462
01/Jul/1995	302	2566	218397
01/Jul/1995	304	3797	0
01/Jul/1995	404	314	0
02/Jul/1995	200	54339	1521111456

Query 2: Top Requested Resources (first 10)

Resource	Requests	Bytes	Hosts
/images/NASA-logosmall.gif	111086	70379226	49583
/images/NASA-logosmall.gif	97267	61055694	41018
/images/KSC-logosmall.gif	89529	92472016	49049
/images/KSC-logosmall.gif	75278	76688780	44912
/images/MOSAIC-logosmall.gif	67349	21158907	30981
/images/USA-logosmall.gif	66968	13561002	30766
/images/WORLD-logosmall.gif	66344	38492922	30488
/images/ksclogo-medium.gif	62663	319890578	26968
/images/MOSAIC-logosmall.gif	60299	19338462	29729
/images/USA-logosmall.gif	59844	12369240	29490

Query 3: Hourly Error Analysis (first 10 rows)

Date	Hour	Errors	Total	Error%	Hosts
01/Aug/1995	0	4	1635	0.24%	4
01/Aug/1995	1	18	1365	1.32%	7
01/Aug/1995	2	5	986	0.51%	2
01/Aug/1995	3	85	1217	6.98%	16
01/Aug/1995	4	1	985	0.10%	1
01/Aug/1995	5	5	1145	0.44%	4
01/Aug/1995	6	1	1020	0.10%	1
01/Aug/1995	7	9	1778	0.51%	6
01/Aug/1995	8	30	2857	1.05%	11
01/Aug/1995	9	11	3199	0.34%	7

MongoDB

```

ETL RUN REPORT

Run ID      : 2
Pipeline    : Pig
Query       : All
Batch ID    : 1
Avg Batch Size : 1730806.00
Records Processed : 3461612
Malformed Records : 8313
Runtime     : 288702 ms
Executed At : 2026-05-17 11:30:34.104873

-- Malformed Records Summary --
Malformed : 8313 / 3461612 (0.24%)

-- Query 1: Daily Traffic Summary (first 10 rows) --

```

Date	Status	Requests	Total Bytes
01/Aug/1995	200	30657	523879212
01/Aug/1995	302	399	29856
01/Aug/1995	304	2608	0
01/Aug/1995	403	1	0
01/Aug/1995	404	242	0
01/Jul/1995	200	57846	1605426462
01/Jul/1995	302	2566	218397
01/Jul/1995	304	3797	0
01/Jul/1995	404	314	0
02/Jul/1995	200	54339	1521111456

```

-- Query 2: Top Requested Resources (first 10) ---

```

Resource	Requests	Bytes	Hosts
/images/NASA-logosmall.gif	111086	70379226	49583
/images/NASA-logosmall.gif	97267	61055694	41018
/images/KSC-logosmall.gif	89529	92472016	49049
/images/KSC-logosmall.gif	75278	76688780	44912
/images/MOSAIC-logosmall.gif	67349	21158907	30981
/images/USA-logosmall.gif	66968	13561002	30766
/images/WORLD-logosmall.gif	66344	38492922	30488
/images/ksclogo-medium.gif	62663	319890578	26968
/images/MOSAIC-logosmall.gif	60299	19338462	29729
/images/USA-logosmall.gif	59844	12369240	29490

```

-- Query 3: Hourly Error Analysis (first 10 rows) --

```

Date	Hour	Errors	Total	Error%	Hosts
01/Aug/1995	0	4	1635	0.24%	4
01/Aug/1995	1	18	1365	1.32%	7
01/Aug/1995	2	5	986	0.51%	2
01/Aug/1995	3	85	1217	6.98%	16
01/Aug/1995	4	1	985	0.10%	1
01/Aug/1995	5	5	1145	0.44%	4
01/Aug/1995	6	1	1020	0.10%	1
01/Aug/1995	7	9	1778	0.51%	6
01/Aug/1995	8	30	2857	1.05%	11
01/Aug/1995	9	11	3199	0.34%	7

Pig

10. Conclusion

The repository presents a clear multi-pipeline ETL prototype for semi-structured web log analytics. It shows a single interface, four execution backends, shared query semantics, and a PostgreSQL-based reporting layer.